

	Departamento: Ciencias de la computacion
	Carrera: Ing. Electronica y Automatizacion Asignatura: Fundamentos de la programacion NRC:28561
	Apellidos y nombres del estudiante: Esteban Daniel Iza Flores

ANTECEDENTES

En la guía práctica se elaboró un ejercicio empleando estructuras repetitivas for y validaciones con MIENTRAS, con el propósito de crear un triángulo rectángulo hueco utilizando asteriscos. Esta actividad permitió reforzar el manejo de ciclos anidados, el uso de condiciones lógicas y la validación de datos tanto en PSeInt como en el lenguaje C.

Para determinar cuándo mostrar un asterisco y cuándo dejar espacios en blanco, se aplicó la condición: $j = 1$, $j = i$ o $i = n$.

Gracias a este ejercicio, se fortaleció la capacidad de razonamiento lógico y la conversión de algoritmos desde pseudocódigo hacia su implementación en Code Blocks.

OBJETIVO

Utilizar estructuras repetitivas FOR y validaciones con REPETIR para el desarrollo de algoritmos. Crear un programa que permita formar un triángulo rectángulo hueco mediante caracteres.

Convertir de manera adecuada un pseudocódigo elaborado en PSeInt al lenguaje C y comprender el funcionamiento del algoritmo a través de una prueba de escritorio.

DESARROLLO

Antes de desarrollar el programa en C, es recomendable plantear la solución en PSeInt, ya que permite representar la lógica del algoritmo con instrucciones similares al lenguaje natural

En este ejercicio se busca generar un triángulo rectángulo hueco de 10 líneas utilizando el asterisco (*)

Pseudocodigo para pseint

Algoritmo Triangulo hueco

Definir fila, columna, altura Como Entero

Repetir

 Escribir "Ingrese la cantidad de lineas del triángulo: "

 Leer altura

 Si altura ≤ 0 Entonces

 Escribir "El numero debe ser mayor que cero."

 FinSi

Hasta Que altura $>$

Para fila $\leftarrow 1$ Hasta altura Con Paso 1 Hacer

 Para columna $\leftarrow 1$ Hasta fila Con Paso 1 Hacer

 Si columna = 1 O columna = fila O fila = altura Entonces

 Escribir Sin Saltar "*" "

 SiNo

 Escribir Sin Saltar " "

 FinSi

FinPara

Escribir ""

FinPara

FinAlgoritmo

Prueba de escritorio en pseint

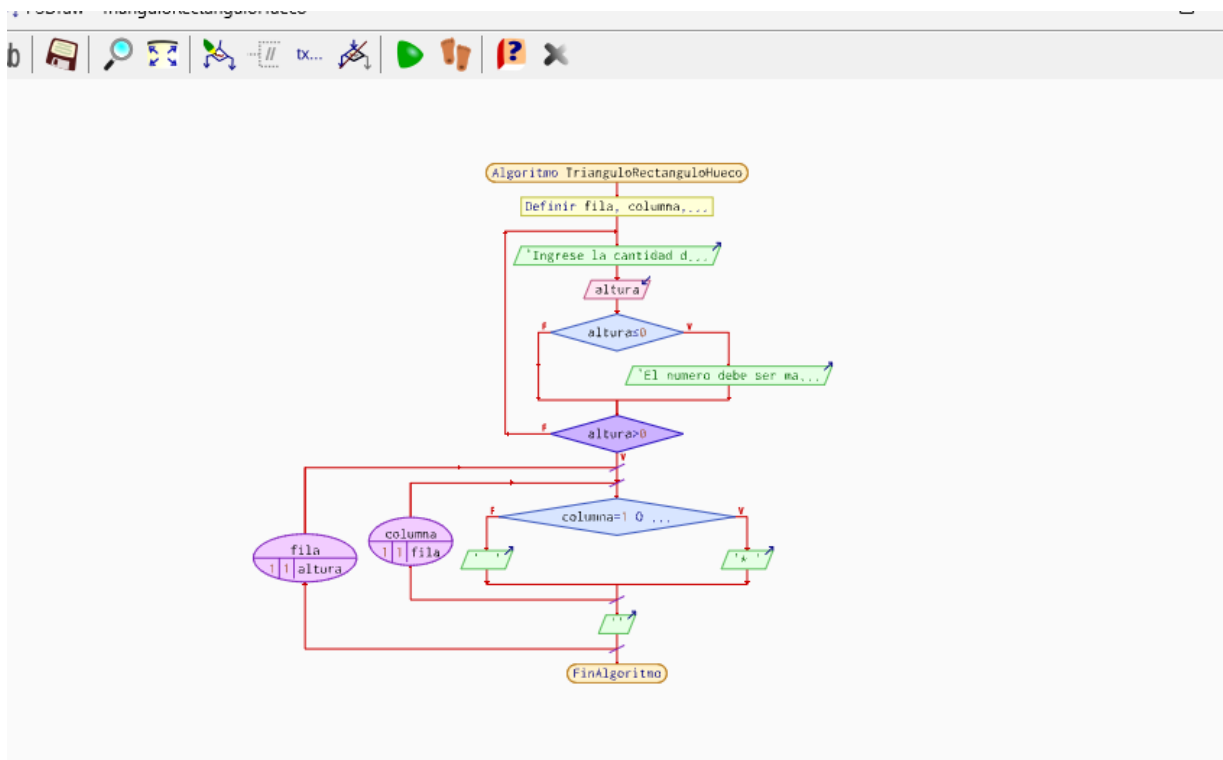
Fila i	Rango de j	Condición que dibuja borde	Salida de la fila	Explicación breve
1	1	$j = 1 \text{ y } j = i$	*	Primera fila: solo un asterisco.
2	1 a 2	$j = 1 \text{ o } j = i$	* *	Tiene borde izquierdo y diagonal.
3	1 a 3	$j = 1 \text{ o } j = i$	* *	El centro queda vacío.
4	1 a 4	$j = 1 \text{ o } j = i$	* *	Aumenta el espacio interno.
5	1 a 5	$j = 1 \text{ o } j = i$	* *	Se mantienen los bordes.
6	1 a 6	$j = 1 \text{ o } j = i$	* *	Continúa el patrón hueco.
7	1 a 7	$j = 1 \text{ o } j = i$	* *	Más columnas internas vacías.
8	1 a 8	$j = 1 \text{ o } j = i$	* *	El borde diagonal se desplaza.
9	1 a 9	$j = 1 \text{ o } j = i$	* *	El triángulo sigue creciendo.
10	1 a 10	$j = 1 \text{ o } j = i \text{ o } i = n$	* * * * * * * * *	Última fila: se completa toda la base.

```

PSeInt - Ejecutando proceso TRIANGULORECTANGULOHUECO
*** Ejecución Iniciada. ***
Ingrese la cantidad de lineas del triangulo:
> -2
El numero debe ser mayor que cero.
Ingrese la cantidad de lineas del triangulo:
> 8
*
* *
*  *
*   *
*    *
*     *
*      *
* * * * * * *
*** Ejecución Finalizada. ***

```

Diagrama de flujo



Codeblocks

PSEINT

Algoritmo TrianguloHueco ...
FinAlgoritmo

Definir fila, columna, altura Como
Entero

Escribir

Leer altura

Repetir ... Hasta Que altura > 0

Si altura <= 0 Entonces

Escribir "El numero debe ser mayor
que cero."

Para fila <- 1 Hasta altura Con Paso
1 Hacer

Para columna <- 1 Hasta fila Con
Paso 1 Hacer

Si columna = 1 O columna = fila O
fila = altura Entonces

Codeblocks

```
int main() { ... return 0; }
```

```
int fila, columna, altura;
```

```
printf();
```

```
scanf("%d", &altura);
```

```
do { ... } while (altura <= 0);
```

```
if (altura <= 0)
```

```
printf("El numero debe ser  
mayor que cero.\n");
```

```
for (fila = 1; fila <= altura;  
fila++)
```

```
for (columna = 1; columna  
<= fila; columna++)
```

```
if (columna == 1
```

Dato importante

Todo programa en C inicia en
main.

Las variables enteras se declaran
con int.

printf permite mostrar
mensajes en pantalla.

& indica la dirección de memoria
de la variable.

El bloque se ejecuta al menos
una vez.

Se valida que el número
ingresado sea correcto.

\n realiza un salto de línea.

El primer FOR controla las filas.

El segundo FOR controla las
columnas.

PSEINT	Codeblocks	Dato importante
O		En C el operador OR se escribe con doble barra vertical.
= para comparar	==	== compara valores; = asigna valores.
Escribir Sin Saltar "*" "	printf("* ");	Imprime el asterisco sin bajar de línea.
Escribir Sin Saltar " "	printf(" ");	Imprime espacios internos para formar el hueco.
Escribir ""	printf("\n");	Permite pasar a la siguiente fila del triángulo.

Código en codeblocks

```
#include <stdio.h>

int main()
{
    int fila, columna, altura;
    do
    {
        printf("Ingrese la cantidad de lineas del triangulo: ");
        scanf("%d", &altura);

        if (altura <= 0)
        {
            printf("El numero debe ser mayor que cero.\n");
        }

    } while (altura <= 0);
    for (fila = 1; fila <= altura; fila++)
    {
        for (columna = 1; columna <= fila; columna++)
        {

            if (columna == 1 || columna == fila || fila == altura)
            {
                printf("* ");
            }
        }
    }
}
```

```

    }
    else
    {
        printf(" ");
    }
}
printf("\n");
}

return 0;
}

```

The screenshot shows a terminal window with the following text:

```

Lineas del triangulo: -5
Ingrese un numero mayor que 0:
Lineas del triangulo: 10
*
* *
*  *
*   *
*    *
*     *
*      *
*       *
*        *
* * * * *

```

Below the pattern, the terminal displays:

```

Process returned 0 (0x0)   execution time : 9.657 s
Press any key to continue.

```

CONCLUSIONES

Las estructuras repetitivas for facilitan el control de filas y columnas para la construcción de figuras usando caracteres.

El uso de condiciones lógicas permite reconocer con mayor claridad los bordes del triángulo hueco.

La prueba de escritorio ayuda a verificar el comportamiento del algoritmo antes de su implementación, analizando cada paso de forma detallada.

RECOMENDACIONES

Para evitar los errores comprobar paso a paso.

Rubrica

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Identifica claramente que el triángulo debe ser hueco, con borde izquierdo, diagonal y base.	Reconoce la forma del triángulo, aunque explica parcialmente la condición de impresión.	Confunde triángulo lleno con triángulo hueco o no distingue los bordes.	4
Seudocódigo en PSeInt	Usa correctamente variables, Repetir, Para, Si/Sino y Escribir Sin Saltar.	Presenta la estructura general con pequeños errores de sintaxis o ubicación.	El pseudocódigo está incompleto o no representa la lógica del ejercicio.	4 pts
Prueba de escritorio	Comprueba el algoritmo fila por fila y explica la salida para $n = 10$.	Realiza una prueba parcial con algunas filas o poca explicación.	No evidencia seguimiento de variables i y j o la salida no coincide.	4 pts
Traducción a C en Code::Blocks	El código compila, ejecuta y mantiene la lógica validada del triángulo hueco.	El código tiene errores menores corregibles, pero la lógica principal es adecuada.	El código no compila o no aplica correctamente FOR, if y operadores lógicos.	4 pts
Presentación y orden	Entrega el trabajo limpio, ordenado y fácil de leer, con comentarios útiles.	El trabajo es comprensible, aunque puede mejorar en orden o comentarios.	El trabajo es difícil de seguir o no evidencia organización.	5 pts